

AI Clarification & Experiment Engine

How to use the LLM meaningfully: structured parsing, missing-information detection, provenance, prompts, and guardrails.

Recommended build: Option 2 - ASK -> CLARIFY -> DEFINE -> TEST -> LEARN

Source basis: the complete 13-page assignment PDF provided by you. Page references inside these guides refer to that assignment.

1. The AI should be a structured reasoning component

The assignment specifically evaluates meaningful AI use rather than a chatbot wrapper. The AI should convert language into a typed experiment draft, identify ambiguity, propose transparent assumptions, and later help explain evidence - but deterministic code should own validation and calculations.

Responsibility split

AI/LLM	Deterministic code
Interpret ambiguous language	Validate fields and ranges
Extract instrument/conditions	Merge clarification answers
Identify missing information	Decide readiness from required fields
Propose assumptions with reasons	Run sample backtest/calculations
Draft hypothesis	Calculate metrics
Explain evidence cautiously	Enforce evidence/conclusion separation

2. Experiment representation with provenance

```
{
  "instrument": { "value": "NIFTY", "source": "user" },
  "timeframe": { "value": "daily", "source": "derived" },
  "entryCondition": {
    "value": "daily_return_pct <= -1.0",
    "source": "assumption",
    "needsConfirmation": true
  },
  "holdingPeriodDays": { "value": null, "source": "missing" },
  "testPeriod": { "value": null, "source": "missing" },
  "costs": { "transactionBps": 10, "slippageBps": 5,
    "source": "assumption", "needsConfirmation": true },
  "hypothesis": "Average forward return after the trigger is positive and exceeds baseline."
}
```

Provenance is a differentiator: the evaluator can immediately see what the user said, what the AI inferred, what it assumed, and what was clarified.

3. Minimum clarification logic

Do not ask ten questions. Ask only what materially changes whether the experiment is meaningful.

Priority	Question	Why it matters
P0	What counts as a "sharp fall"?	Without a numeric threshold the trigger cannot be tested.
P0	How long should the position be held / what is the exit?	Defines outcome measurement.
P1	What historical test period?	Needed to define evidence scope.
P1	When is the trade entered? close vs next open?	Prevents look-ahead ambiguity.
P1	Should costs/slippage be included?	Can change weak edges materially.

Priority	Question	Why it matters
P2	Any volatility/regime filter?	Optional refinement rather than core requirement.

4. Prompt design

System prompt goals

- You are a research experiment structurer, not a trading advisor.
- Never claim profitability or recommend a trade.
- Extract only what the user stated.
- Mark missing important fields explicitly.
- Assumptions must be visible, conservative and confirmable.
- Return JSON that matches the supplied schema.
- Do not fabricate historical results or market data.
- Distinguish user text, derived interpretation and assumption.

Parser prompt template

```

SYSTEM:
Transform a market research question into a structured experiment draft.
Never hide assumptions. If a parameter is necessary for testing and absent,
put it in missingFields. You may propose a suggested value, but label it as
an assumption requiring confirmation. Do not generate market results.

USER QUESTION:
{{query}}

EXISTING CLARIFICATIONS:
{{clarifications}}

RETURN:
- experimentDraft
- missingFields[] { key, question, whyImportant, suggestedValue? }
- assumptions[] { field, value, reason }
- readyToTest: boolean

```

5. Guardrails and failure handling

- If the LLM output fails schema validation, retry once with the validation error summary; then fall back to a safe UI error.
- If instrument is unknown, ask for clarification rather than inventing a symbol.
- If the query is not a researchable market question, explain what is missing and show an example.
- If a user asks for direct financial advice, keep the prototype framed as research/education.
- Never let the LLM generate test metrics; only code/data generates metrics.

Meaningful AI without unnecessary agents

You do not need LangChain, LangGraph, multi-agent orchestration, vector databases or tool-calling for this assignment. Adding them without a real need increases explanation burden and can look like copied complexity. A single structured-output LLM call plus deterministic validation is easier to defend.

6. AI Usage Note content

- Tools used: e.g. Antigravity/Gemini for planning and UI iterations; Cursor/Copilot for implementation suggestions; ChatGPT for reviewing edge cases.
- What you used them for: component scaffolding, schema alternatives, tests, copy review.
- Decisions you made yourself: state machine, provenance model, clarification priorities, metric choice, data labeling, security boundaries.
- Suggestions you rejected: e.g. adding auth, live brokerage integration, LangChain or a database before the core flow.
- What you are proud of: transparent ambiguity handling and separation of evidence from interpretation.

Confirmed Implementation Stack

This page supersedes any earlier wording that presented the AI provider or database as optional choices. For this submission, the selected implementation is fixed:

Layer	Selected technology	How it is used
AI / LLM	Google Gemini API	Parse natural-language research questions, identify ambiguity, propose clarification questions
Database	Neon PostgreSQL	Persist research sessions, structured experiments, clarification answers, test runs, and con
ORM	Drizzle ORM	Typed schema, migrations, and database access from the Next.js server layer.
Validation	Zod	Validate both user input and Gemini structured output before persistence or testing.
Frontend / Backend	Next.js + TypeScript	App Router UI plus server-side route handlers/server functions.

Environment variables

```
GEMINI_API_KEY=...  
DATABASE_URL=postgresql://...  
GEMINI_MODEL=gemini-2.5-flash
```

Recommended data flow

Question -> server validation -> Gemini structured interpretation -> Zod validation -> clarification/confirmation -> experiment persisted to Neon -> deterministic TypeScript test -> result persisted -> Learn view.

Security rules

Never expose GEMINI_API_KEY or DATABASE_URL in client code. Keep both outside NEXT_PUBLIC variables. Validate request sizes, parse Gemini output against a strict schema, use parameterized ORM queries, add API rate limiting, and never let the LLM fabricate backtest metrics.

Suggested Neon schema

research_sessions: id, original_question, status, created_at
experiments: id, session_id, instrument, timeframe, entry_condition, exit_condition, holding_period, test_period, filters_json, costs_json, hypothesis, provenance_json
test_runs: id, experiment_id, dataset_source, metrics_json, warnings_json, created_at

Why this choice is strong for the assignment: Gemini is used for language understanding and ambiguity handling, while Neon stores reproducible research state. Numerical evidence remains deterministic and auditable instead of being invented by the model.